

PROCESAMIENTO DIGITAL DE IMÁGENES

Nombre:

Roberto Reyes Cruz

Enrique Ramirez

Especialidad:

Fecha de la práctica:

Nombre de la práctica: Transformación cilíndrica.

Resultados de aprendizaje Propuestos (RAP's)

Comprende los principios básicos de la transformación cilíndrica y la transformación elíptica.

Analiza y aplica el algoritmo de transformación cilíndrica a una imagen.

Analiza y aplica el algoritmo de transformación elíptica a una imagen.

Comprueba de forma experimental los efectos de aplicar en diferente dimensión la transformación cilíndrica.

Comprueba de forma experimental el efecto de aplicar diferente radio "a" la transformación elíptica.

Objetivo

(Qué producto se espera)

Introducción

La transformación cilíndrica es una técnica de procesamiento de imágenes que se utiliza para corregir la distorsión de la perspectiva en imágenes que contienen objetos cilíndricos, como edificios, tuberías, barriles, entre otros.

En la transformación cilíndrica, la imagen se proyecta sobre un cilindro imaginario que se coloca alrededor del objeto cilíndrico de la imagen. El cilindro se desenrolla en una imagen plana, lo que permite que la imagen se corrija de forma que los objetos cilíndricos parezcan rectos y sin distorsión.

Desarrollo

1. Selecciona una imagen arbitraria.
2. Mediante un programa:
 - a. Carga las imágenes en python.
 - b. Realiza la transformación cilíndrica X e Y a diferentes cuadrantes de la imagen: transformación en X a los cuadrantes I y IV, mientras que transformación en Y a los cuadrantes II y III:

I	II
III	IV

Cuadrantes de la imagen

- c. Realiza 3 transformaciones elípticas en X a la imagen, una con $a=1$, $a=0.5$ y $a=1.5$.
- d. Realiza 3 transformaciones elípticas en Y a la imagen, una con $a=1$, $a=0.5$ y $a=1.5$.

(Muestra el proceso, programa y resultados)

Conclusiones

la transformación cilíndrica es una técnica de procesamiento de imágenes que se utiliza para corregir la distorsión de la perspectiva en imágenes que contienen objetos cilíndricos. Este proceso implica la proyección de la imagen sobre un cilindro imaginario y la aplicación de una transformación de mapeo de píxeles para corregir la distorsión de perspectiva en la imagen original. La transformación cilíndrica se utiliza en diversas aplicaciones, incluyendo la realidad aumentada, la cartografía, la fotografía y la videografía.

Código

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

imagen = cv2.imread("hamster1.jpg")

h, w = imagen.shape[:2]

# Definir puntos de división

x_division = int(w/2)

y_division = int(h/2)

# Definir coordenadas de cada cuadrante

cuadrante_1 = (0, 0, x_division, y_division)

cuadrante_2 = (x_division, 0, w, y_division)

cuadrante_3 = (0, y_division, x_division, h)

cuadrante_4 = (x_division, y_division, w, h)
```

```

def transformacion_cilindrica(imagen, x_coords, y_coords):

    # Obtener el tamaño de la imagen

    h, w = imagen.shape[:2]

    # Definir el radio de la imagen cilíndrica

    f = int(w/2)

    # Definir el centro de la imagen

    c_x, c_y = int(w/2), int(h/2)

    # Crear una imagen vacía para la transformación

    imagen_transformada = np.zeros_like(imagen)

    # Aplicar la transformación en X e Y a los cuadrantes indicados

    for x1, y1, x2, y2 in [x_coords, y_coords]:

        for y in range(y1, y2):

            for x in range(x1, x2):

                if x < c_x:

                    x_new = int(f * np.arctan((x - c_x) / f) + c_x)

                else:

                    x_new = int(f * np.arctan((x - c_x) / f) + c_x)

            denominator = np.sqrt(f*2 - (x - c_x)*2)

            if denominator > 0 and y <= int(f * (y - c_y) / denominator + c_y):

                x_new = int(f * np.arctan((x - c_x) / f) + c_x)

                y_new = int(f * (y - c_y) / denominator + c_y)

```

```

        # Copiar el valor de píxel original en la imagen transformada

        if 0 <= x_new < w and 0 <= y_new < h:

            imagen_transformada[y, x] = imagen[y_new, x_new]

    return imagen_transformada

# Aplicar transformación cilíndrica en X a los cuadrantes 1 y 4
transformacion_x = transformacion_cilindrica(imagen, cuadrante_1, cuadrante_4)

# Aplicar transformación cilíndrica en Y a los cuadrantes 2 y 3
transformacion_y = transformacion_cilindrica(imagen, cuadrante_2, cuadrante_3)

# Definir factores de escala para las transformaciones elípticas en X
a_values = [1, 0.5, 1.5]

# Aplicar transformaciones elípticas en X
for a in a_values:

    transformacion_eliptica_x = cv2.resize(imagen, None, fx=a, fy=1,
    interpolation=cv2.INTER_LINEAR)

    # Mostrar imagen resultante

    plt.imshow(transformacion_eliptica_x)

    plt.show()

# Definir factores de escala para las transformaciones elípticas en Y
b_values = [1, 0.5, 1.5]

```

```
# Aplicar transformaciones elípticas en Y
```

```
for b in b_values:
```

```
    transformacion_eliptica_y = cv2.resize(imagen, None, fx=1, fy=b,  
    interpolation=cv2.INTER_LINEAR)
```

```
# Mostrar imagen resultante
```

```
plt.imshow(transformacion_eliptica_y)
```

```
plt.show()
```

```
# Mostrar imagen resultante de la transformación cilíndrica en X
```

```
plt.imshow(transformacion_x)
```

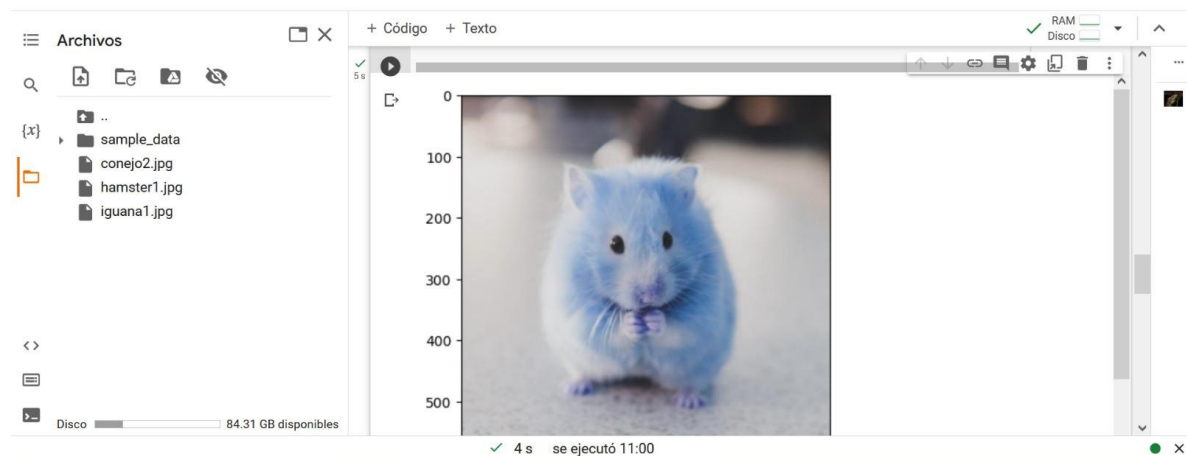
```
plt.show()
```

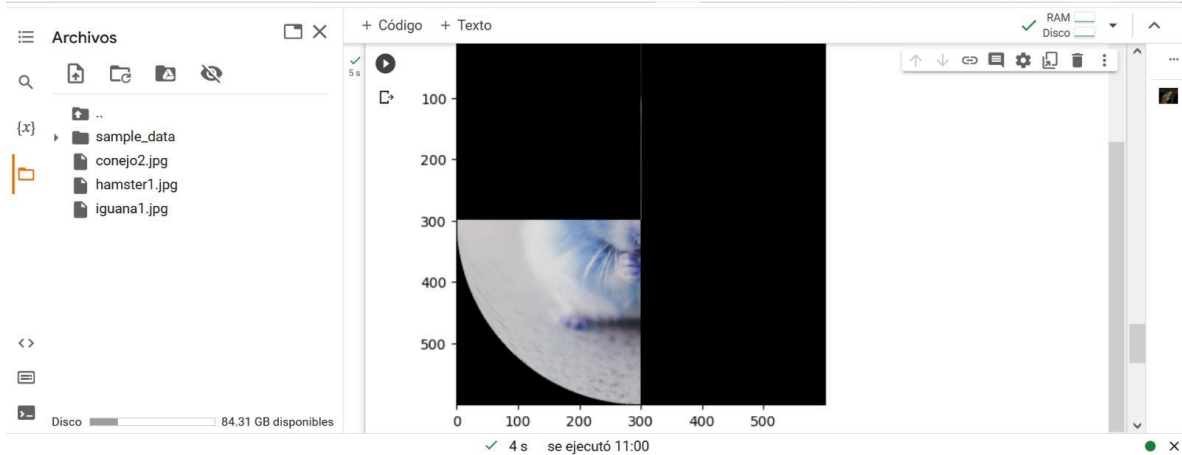
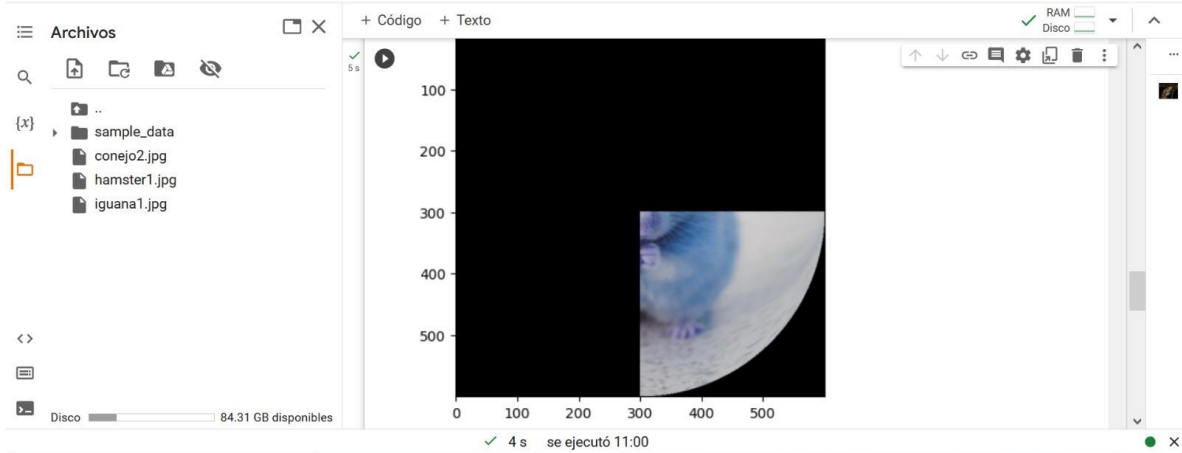
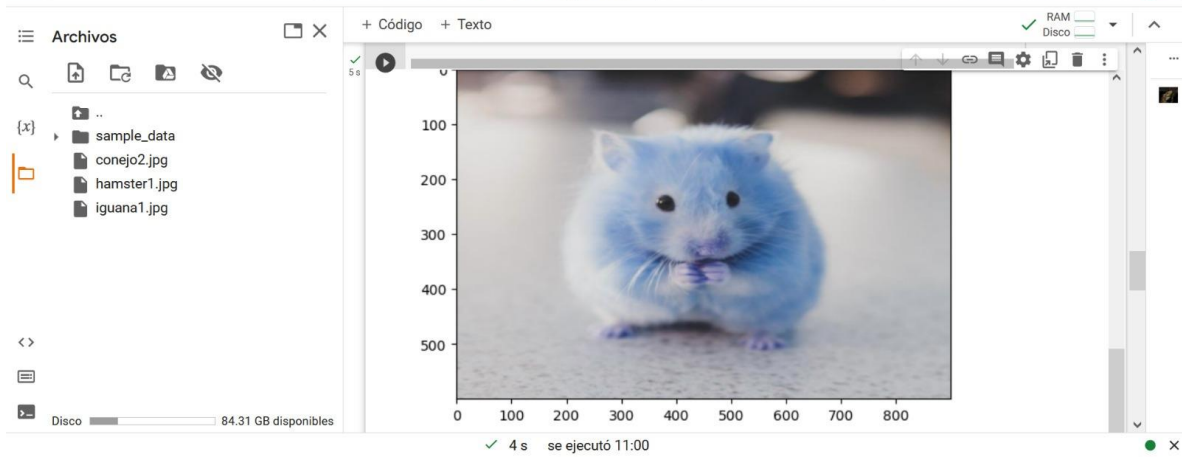
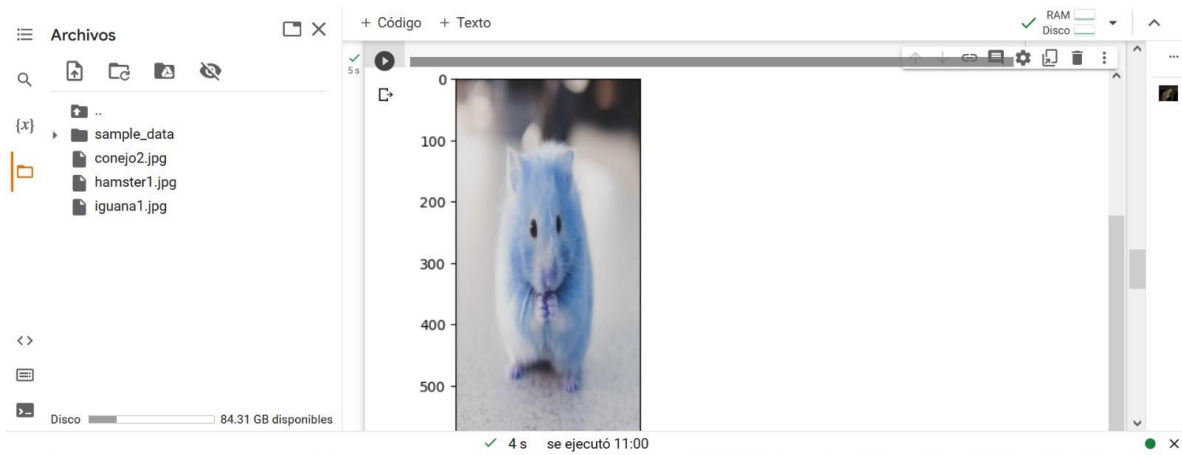
```
# Mostrar imagen resultante de la transformación cilíndrica en Y
```

```
plt.imshow(transformacion_y)
```

```
plt.show()
```

Resultados





Bibliografia

Szeliski, R. (2010). Computer vision: algorithms and applications. Springer Science & Business Media.

Forsyth, D. A., & Ponce, J. (2011). Computer vision: a modern approach. Prentice Hall.

Gonzalez, R. C., Woods, R. E., & Eddins, S. L. (2003). Digital image processing using MATLAB. Prentice Hall.